



IA3 Traceability Card

Student name: Kwashi Elorm Frema

Student ID: 22397307

Group Repo Link: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-33-merakiartsafrica>

Individual Repository Link: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-individual-engineering-evidence-portfolio-2026-Elena-Nsokie18.git>

Team name: SEG26-33-MerakiArtsAfrica

Project title: Meraki Arts Africa Website

Date submitted: 13 July 2026

AI disclosure statement : I did not use AI assistance. I understand that I must be able to explain and defend every part of this submission.





1. Selected use case

Use case name:

View Gallery

Source evidence: Client requirement requesting a gallery to showcase Meraki Arts Africa's projects, events and community activities. (Replace this with your actual client meeting note, requirement ID, or GitHub issue URL.)

Why this use case matters to the client:

The gallery allows visitors to view photographs of Meraki Arts Africa's projects, events and activities. It helps demonstrate the organization's impact, promotes its work, and encourages visitors to support or partner with the foundation.

2. Use case card

Actor: Website Visitor

Goal: View images and information in the gallery.

Trigger: The visitor selects the Gallery option from the navigation menu.

Preconditions:

The website is available.

Gallery images have been uploaded.

The visitor has internet access.

The Gallery page loads successfully.

Main success flow:

The visitor opens the Meraki Arts Africa website.

The visitor clicks Gallery.

The system retrieves gallery items from the database.

The system displays the gallery images and their information.

The visitor scrolls through and views the gallery content.

Alternate flow:

The visitor selects a gallery image.

The system opens a larger version of the image together with its description.

Exception/failure flow:

The visitor opens the Gallery page.

The system cannot retrieve gallery data because of a server or database error.

The system displays an appropriate error message or "No gallery items available."

The visitor remains on the Gallery page without the application crashing.

Postconditions:

The gallery has been displayed successfully, or

The visitor receives an appropriate message if the gallery cannot be loaded.

3. User story

As a website visitor, I want to view the gallery so that I can see Meraki Arts Africa's projects, events and community impact.

4. Acceptance criteria

- AC1 – Successful path

Given gallery items are available,

When the visitor opens the Gallery page,

Then the system displays all available gallery images together with their titles or descriptions.

- AC2 – Error path

Given the gallery data cannot be retrieved,

When the visitor opens the Gallery page,

Then the system displays an appropriate error message without crashing.

5. Non-functional requirement link

NFR category: Performance

NFR statement: The Gallery page should load within three seconds under normal network conditions.

Why it matters: Visitors are more likely to remain on the website and explore Meraki Arts Africa's work if gallery images load quickly. Slow loading can reduce user engagement and negatively affect the user experience.

6. GitHub issue link

Issue title: Gallery Feature

Issue URL: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-33-merakiartsafrica>

7. Test idea

Test name: Display Gallery Images Successfully

Input/context: A visitor opens the Gallery page while gallery images exist in the database.

Expected result: The system retrieves and displays all gallery images and related information correctly.

8. Request-flow thinking

Device/interface: Web browser

Route/screen/endpoint: Gallery page → GET /api/gallery

Data sent: HTTP GET request to retrieve gallery items.

Validation point: The server verifies that the request is valid and checks whether gallery records exist.

Database/external service touched: MongoDB Gallery collection.

Response: The server returns the gallery data, and the frontend displays the images.

Failure point: If the database or server is unavailable, the API returns an error and the frontend displays an error message.

Security/privacy concern: Only approved gallery content should be displayed to visitors.

Administrative functions such as uploading, editing or deleting gallery items must be restricted to authorized users.

7. Request-flow questions to answer

- What device or interface is the actor using? = Laptop and mobile versions
- Is the action performed through a browser, mobile app, desktop app, command or internal admin interface? = browser
- What screen, route, command or endpoint is involved? = GET /api/gallery
- What data is sent? = A GET request to retrieve gallery images and their details
- What validates the request? = The server validates the request and checks that the gallery endpoint is available before retrieving data.
- What database table, collection, file or external service is touched? = MongoDB **Gallery** collection.
- What response comes back to the user? = The server returns the gallery data (images, titles, descriptions), and the frontend displays them on the Gallery page.
- What happens if validation fails? = If the request is invalid or the database is unavailable, the server returns an error, and the user sees an appropriate error message
- What happens if the database or external service is unavailable? = If the request is invalid or the database is unavailable, the server returns an error, and the user sees an appropriate error message
- What data or action must be protected? = Only approved gallery content should be accessible to visitors. Administrative actions such as uploading, editing, or deleting gallery items must be restricted to authenticated administrators.